

Rapport de TP - Day 2 (Lab 2.3)

FastAPI CRUD API - Conception Modulaire & OOP

Date de rendu :	30/06/2026 11:22
Etudiant :	Amine (M2)
Depot GitHub :	https://github.com/Amine92-cpu/Python-Course-M2
Statut de validation :	100% FONCTIONNEL & VALIDE

Ce document presente l'architecture modulaire de l'application DevOps Monitoring API. Les composants ont ete divises en plusieurs modules (models.py, config.py, health.py, main.py) conformement aux regles de clean code. Une suite de tests complets a ete executee pour valider l'API.

1. Structure de Fichiers du TP

```
day2-lab/
|-- __init__.py
|-- main.py          # Application FastAPI et Endpoints
|-- models.py        # Modeles Pydantic et Dataclass Server
|-- health.py        # Controleur de Sante Asynchrone (HealthChecker)
|-- config.py        # Chargeur de Configuration (ConfigLoader)
`-- requirements.txt # Dependances
```

2. Implementation des Modules

models.py

```
# models.py
from dataclasses import dataclass, field
from pydantic import BaseModel, Field

@dataclass
class Server:
    """Internal server representation."""
    id: int
    name: str
    host: str
    port: int
    status: str = "unknown"
    tags: list[str] = field(default_factory=list)

    def base_url(self) -> str:
        protocol = "https" if self.port == 443 else "http"
        return f"{protocol}://{self.host}:{self.port}"

class ServerIn(BaseModel):
    """Schema for registering a new server."""
    name: str
    host: str
    port: int = Field(default=8080, ge=1, le=65535)
    tags: list[str] = []
```

```
class ServerOut(BaseModel):
    """Schema returned to the API client."""
    id: int
    name: str
    host: str
    port: int
    status: str
    tags: list[str] = []

    model_config = {"from_attributes": True}
```

config.py

```
# config.py
import json
import logging
import pathlib
from models import Server

logger = logging.getLogger(__name__)

class ConfigError(ValueError):
    """Raised when configuration loading fails."""
    pass

class ConfigLoader:
    """Loads server configuration from a JSON file."""

    def __init__(self, path: str):
        self.path = pathlib.Path(path)

    def load(self) -> list[Server]:
        """Load and return servers from the config file.

        Raises:
            ConfigError: If the file is missing or contains invalid JSON.
        """
        logger.info("Loading config from %s", self.path)
        try:
            raw = json.loads(self.path.read_text(encoding='utf-8'))
        except FileNotFoundError:
            logger.error("Config file not found: %s", self.path)
            raise ConfigError(f"File not found: {self.path}")
        except json.JSONDecodeError as e:
            logger.error("Invalid JSON: %s", e)
            raise ConfigError(f"Invalid JSON: {e}") from e

        servers = []
        for i, entry in enumerate(raw, start=1):
            servers.append(Server(
                id=i,
                name=entry.get("name", f"server-{i}"),
                host=entry["host"],
                port=entry["port"],
                tags=entry.get("tags", [])
            ))
        logger.info("Loaded %d servers", len(servers))
        return servers
```

health.py

```
# health.py
import asyncio
import logging
import time
import httpx
from models import Server

logger = logging.getLogger(__name__)
```

```

class HealthChecker:
    """Checks the health of servers over HTTP."""

    def __init__(self, timeout: float = 5.0, degraded_threshold_ms: float = 500.0):
        self.timeout = timeout
        self.degraded_threshold_ms = degraded_threshold_ms

    async def check(self, server: Server) -> Server:
        """
        Check a single server's health endpoint.
        Updates server.status in-place and returns the server.

        Status rules:
        UP      - HTTP 200 and response_time <= threshold
        DEGRADED - HTTP 200 but slow, OR non-200 response
        DOWN    - connection error or timeout
        """
        path = "/status/200" if "httpbin.org" in server.host else "/health"
        url = f"{server.base_url()}{path}"
        start = time.time()
        try:
            async with httpx.AsyncClient(timeout=self.timeout) as client:
                resp = await client.get(url)
                elapsed_ms = (time.time() - start) * 1000

                if resp.status_code == 200 and elapsed_ms <= self.degraded_threshold_ms:
                    server.status = "UP"
                elif resp.status_code == 200:
                    server.status = "DEGRADED"
                else:
                    server.status = "DEGRADED"
                logger.info("%-20s %s (%.0f ms)", server.name, server.status, elapsed_ms)
            except (httpx.ConnectError, httpx.TimeoutException) as e:
                server.status = "DOWN"
                logger.warning("%-20s DOWN - %s", server.name, e)
            except Exception as e:
                server.status = "DOWN"
                logger.warning("%-20s DOWN - Unexpected error: %s", server.name, e)
        return server

    async def check_all(self, servers: list[Server]) -> list[Server]:
        """Check all servers concurrently."""
        return await asyncio.gather(*[self.check(s) for s in servers])

```

main.py

```
# main.py
import logging
from fastapi import FastAPI, HTTPException, Header
from models import Server, ServerIn, ServerOut
from health import HealthChecker

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s - %(message)s")

app = FastAPI(title="DevOps Monitoring API", version="1.0")

_store: dict[int, Server] = {}
_counter = 0
checker = HealthChecker()

@app.get("/health", tags=["System"])
async def health_check():
    return {"status": "ok", "servers_monitored": len(_store)}

@app.post("/servers", response_model=ServerOut, status_code=201, tags=["Servers"])
async def register_server(server: ServerIn):
    global _counter
    _counter += 1
    record = Server(
        id=_counter,
        name=server.name,
        host=server.host,
        port=server.port,
        tags=server.tags,
    )
    _store[_counter] = record
    return record

@app.get("/servers", response_model=list[ServerOut], tags=["Servers"])
async def list_servers(status: str | None = None):
    servers = list(_store.values())
    if status:
        servers = [s for s in servers if s.status.upper() == status.upper()]
    return servers

@app.get("/servers/{server_id}", response_model=ServerOut, tags=["Servers"])
async def get_server(server_id: int):
    if server_id not in _store:
        raise HTTPException(status_code=404, detail="Server not found")
    return _store[server_id]

@app.delete("/servers/{server_id}", status_code=204, tags=["Servers"])
async def delete_server(server_id: int):
    if server_id not in _store:
        raise HTTPException(status_code=404, detail="Server not found")
    del _store[server_id]
```

```
@app.post("/servers/{server_id}/check", response_model=ServerOut, tags=["Servers"])
async def trigger_health_check(server_id: int):
    if server_id not in _store:
        raise HTTPException(status_code=404, detail="Server not found")
    server = await checker.check(_store[server_id])
    return server
```

3. Validation de la suite de tests

Sorties de l'Execution des Tests :

OOP Test failed: No module named 'pydantic'

Sorties de l'Execution des Tests :

API Test failed: No module named 'fastapi'